



山东大学
SHANDONG UNIVERSITY

SOAR 2026 MiniCPM-SALA 推理优化技术复盘

模型特点、方法空间、决策依据与失败复盘

SOAR 2026 参赛复盘

MiniCPM-SALA / SGLang / W4A16 / FP8KV

2026-06-08



1. 先讲模型：为什么 SALA 不是普通 Transformer	
1 模型特点决定优化边界	2
运行链路有多层系统边界	3
评分约束改变技术路线	4
2. 量化方法选择：先讲方法空间，再讲为什么选	
GPTQ 可选量化方法：RTN / GPTQ / AWQ / Mixed	
precision 为什么不是 RTN：它证明了 W4 可跑，也证明了	6
自己为啥不选 GPTQ：失败逐步可解释，修复后确实	
对模型选择：不是问“要不要 W4”，而是问“哪里能	
过 gate W4”	9
MLP-only / Full W4 / Selective BF16 的取舍	10
为什么先做 MLP-only	11
Full W4A16 的真实结论：有速度，但不是当前最	
4 优 Runtime 与 artifact：这些不是细节，是成败边	
界	13
Runtime dtype：bf16 是 W4A16 成功拐点	14

Artifact 语义：早期失败大量来自这里	15
5. KV cache：FP8KV 是高潜力路线，但当前实现	
没打对	16
KV cache 方法空间	17
FP8KV 现有结果说明：方向能跑，收益没兑现	18
FP8KV 下一步应该怀疑哪些细节	19
6. Prefill、fusion 与验证体系	20
Chunked prefill：是性能 knob，但内存预算会被	
“Kernel fusion” SALA residual 不能套通用 Llama	21
模型	
验证体系：每层验证回答不同问题	22
7. 结论与建议	24
如果重新做一次：推荐实验顺序	25
最终汇报结构建议	25
结束页	25

01 1. 先讲模型：为什么 SALA 不是普通 Transformer

核心判断

MiniCPM-SALA 的推理优化不是“选一个量化算法”这么简单。它是一个 hybrid long-context runtime:

- full / sparse attention 与 lightning attention 混合;
- dense 层和 lightning 层走不同 cache / state 路径;
- sparse backend 依赖 InfLLM v2 与定制 CUDA kernel;
- residual scaling 带有 $\text{scale_depth} / \sqrt{L}$, 不能套通用 Llama fusion。

结构	运行时含义	优化影响
Full/Sparse attention	RadixAttention + FlashInfer	KV cache / qkv / RoPE 敏感
Lightning attention	SimpleGLA recurrent state	不等价于普通 KV cache
MLP	gate_up_proj + down_proj	W4A16 权重带宽主要收益源
SALA residual	residual + hidden * scale_depth/sqrt(L)	fusion 必须保语义

从提交包到 CUDA kernel

submission.tar.gz → prepare_env / prepare_model
→ HF artifact → SGLang loader → MiniCPM backend
→ CUDA kernels

这条链路上任何一层语义错位, 都会表现成“模型坏了”:

- tokenizer 被重写;
- config class 不一致;
- qzeros 编码不匹配;
- dynamic skip 规则不匹配;
- dtype 边界不一致;
- FlashInfer JIT cache 不可迁移。

边界	典型失败	教训
Tokenizer	local 非零 / platform 0	GPTQModel save 会重写 chat template
Config	model_type / AttributeError	AutoConfig 与 SGLang config 必须对齐
Quant artifact	acc=0	qzeros=0x77 必须修成 Marlin 0x88
Runtime dtype	CUDA graph dtype mismatch	bf16 runtime 是正确边界
JIT cache	prepare 卡死 / Ninja path	不要跨机器携带 cache

Variant	acc_ori	final_score	S1	含义
BF16 baseline	–	19.13	–	保底可运行
v5j_dtype_bf16	82.18	23.18	717.76	W4A16 质量 anchor
chunk32k_safe	80.31	22.90	720.76	安全但无明显收益
full_w4a16	78.27	22.85	621.91	快但 acc 损失抵消收益
FP8KV DENSEQKV	78.67	20.87	655.70	可跑但细节未兑现收益

决策原则

平台分数先受正确性 gate 约束，再看性能。

所以比赛中每个方法都要问三件事：

1. 它能不能稳定过 gate?
2. 它是否真的命中当前 runtime 的主瓶颈?
3. 它是否值得消耗一个 platform slot?

02

2. 量化方法选择: 先讲方法空间, 再讲为什么选

GPTQ

可选量化方法: RTN / GPTQ / AWQ / Mixed precision

方法	基本思路	优点	缺点 / 风险	本比赛选择
RTN	round-to-nearest; 无 Hessian	实现简单; artifact 可控; 适合摸格式	4-bit 误差大; SALA 上质量不足	诊断工具, 不作为主线
GPTQ	Hessian-aware PTQ; 逐层误差补偿	质量潜力高; SGLang 有 gptq_marlin 路径	GPTQModel 与 SALA loader/env 兼容复杂	主线; 修复后过 gate
AWQ	activation-aware; 保护 outlier channel	理论适合 SALA activation outlier	llm-compressor / compressed-tensors 另开工程链	高潜力备选, 不是最快闭环
Mixed precision	敏感模块 BF16, 其余 W4A16	贴合 SALA 分层敏感性	需要 artifact surgery 和严格 loader 对齐	后期 selective 主方向

为什么不是 RTN: 它证明了 W4 可跑, 也证明了自己不够

RTN 的价值

RTN 在早期的作用是摸清底层格式, 而不是最终冲分。

它帮助确认:

- Marlin 只接受 `sym=True` 的 `uint4b8`;
- `scale` 应除以 7 而不是 8;
- 4-bit 不是必然 `acc=0`;
- 但是无 Hessian 的误差对 SALA 仍过大。

实验	结果	解释
RTN <code>sym=False</code>	startup reject	Marlin config 不支持
RTN <code>scale /8</code>	<code>acc_ori</code> ≈42.51	<code>scale</code> 公式错但仍非零
RTN <code>scale /7</code>	<code>acc_ori</code> ≈42.18	修格式不提升质量

结论 RTN 是格式诊断工具; 主线必须转向 Hessian-aware 的 GPTQ 或 activation-aware 的 AWQ。

GPTQ 不是一开始就赢

早期 GPTQModel 一路失败, 但失败逐步从“模型坏了”变成可定位的工程问题:

- SUPPORTED_MODELS 注册快照;
- qzeros 0x77 -> 0x88;
- tokenizer 被 GPTQModel 重写;
- transformers / flash_attn / torchao 版本;
- dynamic skip 与 SGLang module name。

阶段	结果	说明
v17/v21/v22	platform 0	qzeros + tokenizer + env 叠加问题
1849 fix stack	acc_ori=46.89	首次非零, 证明方向成立
bf16 runtime	acc_ori=82.18	首次稳定过 gate

选择理由 GPTQ 的失败可以被单变量修复和验证; AWQ 虽有潜力, 但当时工程闭环更长。

03

3. 模块选择: 不是问“要不要 W4”, 而是问“哪里能 W4”

MLP-only / Full W4 / Selective BF16 的取舍

方法	基本思路	优点	缺点 / 风险	本比赛选择
MLP-only W4	只量化 MLP; attention 保 BF16	风险最低; 仍有权重带宽收益	attention GEMM 加速拿不到	最早稳定过 gate, 质量 anchor
Full W4A16	MLP + q/k/v/o 全量化	S1 明显变快	acc 损失 2-4pp, 容易跌 gate	证明可运行, 但 tradeoff 不优
Selective BF16	只恢复 late lightning / qkv / o_proj 等敏感模块	可精细调 accuracy-speed 边界	组合多, 必须单变量队列	后期最符合 SALA 结构
Lightning-skip	Lightning 层 MLP 回 BF16	针对 recurrent 长程误差	safetensors 物理 key 与 loader 易错	思路合理, 实现需重写 shard

当时的核心假设

v17 full-attn GPTQ 能启动但 acc=0。可能原因有两个：

- attention projection W4A16 破坏长上下文检索与停止信号；
- 或者 artifact / runtime 兼容链仍然坏。

MLP-only 是最小化变量的选择：保 attention BF16，仍然压缩大量 MLP 权重。

如果结果	说明	下一步
MLP-only 过 gate	attention 可能是敏感区	再逐步加 attention quant
MLP-only 仍 0	问题更可能在 artifact/ env/tokenizer	先修工程链
full-attn 后来 78.27	attention quant 可跑但伤 质量	改 selective BF16

Full W4A16 的真实结论: 有速度, 但不是当前最优 tradeoff

Variant	acc_ori	final_score	S1
v5j_dtype_bf16	82.18	23.18	717.76
chunk32k_safe	80.31	22.90	720.76
full_w4a16	78.27	22.85	621.91

解释

Full W4A16 的 S1 明显更好, 说明 attention-GEMM 加速真实存在。

但 acc_ori 下降到 78.27, 低于 MLP-only 和 chunk32k_safe, 最终分数被准确率损失抵消。

这不是 full W4 被否定, 而是说明需要 selective BF16 或更好的 scale 校准, 而不是直接全量化。

04 4. Runtime 与 artifact: 这些不是细节，是成败边界

Runtime dtype: bf16 是 W4A16 成功拐点

方法	基本思路	优点	缺点 / 风险	本比赛选择
fp16 sed-patch	把 SALA backend 文本替换成 fp16	表面上对齐 Marlin fp16 输出	破坏 sparse backend bf16 假设; query/key mismatch	被平台 crash 否定
bf16 runtime	--dtype bfloat16; 让 SGLang 接回 bf16	保持 sparse backend dtype 一致	需要确认 Marlin output cast 行为	平台 acc_ori=82.18, 最终采用

Artifact 语义: 早期失败大量来自这里

问题	表面现象	真实根因	修复策略
qzeros	GPTQ acc=0	GPTQModel 写 0x77; Marlin 期望 0x88	post-save rewrite + inspect
Tokenizer	local/platform 不一致	GPTQModel 重写 chat_template; 旧 transformers 忽略 sidecar	强制复制 base tokenizer
Config	model_type / AttributeError	AutoConfig 与 MiniCPMHybridConfig 冲突; 写入只读派生字段	删除 AutoConfig 和 derived keys
Dynamic rules	KeyError / load mismatch	GPTQModel 与 SGLang 模块名不同	规则、index、物理 shard 同步
Install/cache	prepare 卡住	网络下载、FlashInfer cache 绝对路径	离线 wheel + timeout + 不搬 cache

05

5. KV cache: FP8KV 是高潜力路线, 但当前实现没
打对

方法	基本思路	优点	缺点 / 风险	本比赛选择
BF16 KV	原始 KV cache 精度	质量稳定; 实现简单	显存和带宽贵	作为稳定 baseline
FP8 E4M3/E5M2 KV	KV cache 低精度存储	理论显存减半; 长上下文潜力大	scale / dtype / backend 细节复杂	高潜力但当前未打通收益
Scalar scale	每层或每 tensor 一个 scale	实现简单; 接近官方路径	粒度粗; 可能压坏 head 分布	已尝试但不够
Per-head/group scale	更细粒度校准 KV range	更贴近 GQA/head 分布	需要 runtime bridge, 易错	后续重点怀疑点
POSTQ scale	从最终 W4 artifact 测 KV scale	测量源更接近服务源	GPTQModel reload gauntlet 复杂	值得继续查细节

FP8KV 现有结果说明: 方向能跑, 收益没兑现

实验	acc_ori	final_score	S1	说明
hardened FP8KV	76.31	0.0	720.98	完整 eval; 质量掉 gate
REALCALIB multi8K	77.33	0.0	650.26	强 replay 仍低
DENSEQKV multi8K	78.67	20.87	655.70	scale-source 有效但不够

解释口径

这些结果不能写成“FP8KV 没前途”。

更准确是: FP8KV 已从 plumbing 问题进入数值与热路径问题。当前实现没有把 KV cache 容量/带宽收益转化为平台速度或分数。

精度细节

- k_scale / v_scale 测量源: BF16 base、DENSEQKV、最终 W4 artifact 哪个更接近服务时分布?
- scalar scale 是否太粗, 需要 per-head / per-group?
- GQA / head-group bridge 是否严格正确?
- e4m3 / e5m2 的 range-precision 是否匹配 SALA activation?

性能细节

- FP8 写入和 attention 读取是否被额外 scale/dequant 抵消?
- dense-as-sparse 是否让 benchmark 走到 FP8KV 覆盖不到的热路径?
- Lightning attention、MLP GEMM、sampling 是否才是主瓶颈?
- local smoke 是否覆盖了真正导致平台掉分的长输出任务?

06 6. Prefill、fusion 与验证体系



Chunked prefill: 是性能 knob, 但内存预算会被“模型答对”改变

方法	基本思路	优点	缺点 / 风险	本比赛选择
chunk 8K	保守 prefill 分块	稳定; v5j 成功	prefill 吞吐空间有限	质量 anchor
chunk 32K	更大 prefill 分块	理论更高吞吐	需要动态内存余量	mem-frac 0.70 safe anchor
chunk 65K	激进 prefill	理论吞吐更高	长输出 activation / workspace OOM	平台 OOM; 不能盲交

候选方法

- RoPE upcast removal;
- RMSNorm fusion;
- fused add_rmsnorm;
- attention backend path tuning.

平台教训

opfusion v1 平台 acc_ori=24.67。

根因: 把 SALA 的 $\text{residual} + \text{hidden} * \text{scale_depth} / \sqrt{L}$ 改成 Llama 类 fused residual path, 单层 CPU 等价测试没有覆盖 32 层长上下文累计误差。

通用 LLM 上成立的 fusion, 在 SALA 上必须重新验证 residual 语义。

验证体系：每层验证回答不同问题

验证层	能证明什么	不能证明什么	本比赛教训
3-prompt endpoint smoke	server/API 基本可跑	不能证明质量	Path Y 可 smoke, 平台仍低分
30-sample local	快速发现明显错误	容易被短任务误导	opfusion smoke 假阳性
60/150 long canary	发现 QA/CWE/NIAH 长输出风险	仍不等于 hidden set	FP8KV 后 30 条掉到 78
platform run	最终 gate + benchmark	成本高, 不适合盲试	必须靠 preflight 降低浪费

07 7. 结论与建议



1. 固定模型语义边界: tokenizer / config / qzeros / dtype / dynamic rules 先做 preflight。
2. 建立质量 anchor: GPTQ MLP-only + bf16 runtime, 先证明 W4A16 能稳定过 gate。
3. 做 selective quant: 围绕 SALA layer type 和 qkv/o_proj/MLP 做单变量边界实验。
4. 深挖 FP8KV: POSTQ scale、per-head/group scale、GQA bridge、dense-as-sparse hot path。
5. 做严格安全 fusion: 只做不改 residual 语义的优化, 并用长输出 canary 验证。

1. 模型特点

hybrid attention / lightning / sparse backend

2. 量化方法

RTN / GPTQ / AWQ / mixed precision

3. 模块选择

MLP-only / full W4 / selective BF16

4. Runtime 工程

dtype / tokenizer / config / qzeros

5. 性能路线

FP8KV / prefill / fusion

6. 验证方法

smoke / canary / platform

MiniCPM-SALA 优化的难点不是“选一个量化算法”，
而是让量化、KV cache、runtime 和验证体系同时与模
型结构对齐。



山东大学

SHANDONG UNIVERSITY

感谢聆听

Thanks for Listening